



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

COMPUTAÇÃO DE ALTO DESEMPENHO

DOCENTE: CARLA DOS SANTOS SANTANA

E SAMUEL XAVIER DE SOUZA

DISCENTE: CLARA VIRGÍNIA NASCIMENTO SANTOS (20240002212)

Tarefa 16

1. ENUNCIADO

Desenvolver um escalonador dinâmico de tarefas utilizando MPI no modelo Líder–Trabalhador. O processo líder será responsável por distribuir tarefas aos trabalhadores de forma dinâmica, enviando novas tarefas à medida que os trabalhadores finalizam as tarefas anteriores. A aplicação do escalonador consistirá na paralelização do cálculo dos números primos em um intervalo de valores.

Fazer avaliação de eficiência e speedup à medida que aumenta a quantidade de tarefas e a quantidade de trabalhadores. Garantir que não haverá deadlock.

2. DISCUSSÃO

Nas tarefas anteriores, a paralelização foi feita utilizando OpenMP, em que as threads eram criadas dentro de um mesmo processo e compartilhavam a memória. Contudo, nesta tarefa, a proposta é diferente, pois agora será utilizado MPI, que trabalha com múltiplos processos. Dessa maneira, cada processo possui sua própria memória e a comunicação entre eles ocorre através de envio e recebimento de mensagens.

O modelo usado nesta tarefa é chamado de Líder–Trabalhador. Nesse modelo, existe um processo principal, chamado de líder, que fica responsável por controlar a distribuição das tarefas. Os demais processos são os trabalhadores, que apenas recebem uma tarefa, executam o cálculo e enviam o resultado de volta ao líder.

Nesse caso, a tarefa escolhida foi contar quantos números primos existem em um intervalo. O intervalo total é dividido em blocos menores, e cada bloco representa uma tarefa. Por exemplo, se o intervalo for de 2 até 10.000.000 e o tamanho da tarefa for 100.000, então o programa terá várias tarefas menores, como 2 até 100.001, depois 100.002 até 200.001, e assim sucessivamente.

A vantagem do escalonamento dinâmico é que o líder não entrega todas as tarefas de uma vez para todos os trabalhadores. Ele entrega uma tarefa inicial para cada trabalhador e, quando algum trabalhador termina, ele recebe outra tarefa. Assim, trabalhadores mais rápidos não ficam parados esperando os mais lentos terminarem.

Isso é importante no cálculo de números primos, pois nem todos os números têm o mesmo custo de verificação. Quanto maior o número, maior pode ser a quantidade de divisões necessárias para descobrir se ele é primo ou não. Portanto, se a distribuição fosse totalmente fixa, um trabalhador poderia receber uma parte mais pesada e demorar muito mais que os outros, causando desequilíbrio de carga.

2.1. MPI

MPI significa Message Passing Interface, ou seja, uma interface para passagem de mensagens. Diferente do OpenMP, em que as threads compartilham memória, no MPI os processos se comunicam explicitamente usando funções como MPI_Send e MPI_Recv.

As principais funções utilizadas na tarefa foram:

- MPI_Init: inicia o ambiente MPI;
- MPI_Comm_rank: identifica o número do processo atual;
- MPI_Comm_size: identifica a quantidade total de processos;
- MPI_Send: envia uma mensagem para outro processo;
- MPI_Recv: recebe uma mensagem de outro processo;
- MPI_Wtime: mede o tempo de execução;
- MPI_Finalize: finaliza o ambiente MPI.

No programa, o processo de rank 0 será o líder. Todos os demais processos, rank 1, rank 2, rank 3, etc., serão trabalhadores.

2.2. Escalonamento Dinâmico

O escalonamento dinâmico funciona da seguinte forma:

- 1º) O líder separa o intervalo total em tarefas menores;
- 2º) O líder envia uma tarefa inicial para cada trabalhador;

3º) Cada trabalhador calcula a quantidade de primos naquele intervalo recebido;

4º) O trabalhador envia o resultado de volta ao líder;

5º) Quando o líder recebe o resultado, ele verifica se ainda existem tarefas pendentes;

6º) Se existir tarefa pendente, o líder envia uma nova tarefa para aquele mesmo trabalhador;

7º) Se não existir mais tarefa, o líder envia uma mensagem de parada;

8º) O trabalhador, ao receber a mensagem de parada, encerra sua execução.

Dessa maneira, o programa fica mais balanceado, pois os trabalhadores recebem novas tarefas conforme terminam as anteriores. Portanto, o trabalho não fica preso em uma divisão fixa feita no começo da execução.

2.3. Speedup E Eficiência

O Speedup é uma forma de analisar se o uso de paralelismo trouxe mais rapidez de processamento ou não. Assim, pode ser calculado pela relação:

- $\text{Speedup} = \text{Tempo Sequencial} / \text{Tempo Paralelo}$
- $\text{Speedup} > 1$, paralelismo ajudou
- $\text{Speedup} = 1$, paralelismo não ajudou
- $\text{Speedup} < 1$, paralelismo piorou
- $\text{Speedup} = \text{Nº de trabalhadores}$, speedup linear, sendo o ideal

A partir dessa relação, é possível calcular a Eficiência:

- $\text{Eficiência} = \text{Speedup} / \text{Nº de trabalhadores}$
- Eficiência = 100%, uso perfeito dos trabalhadores
- Eficiência alta indica bom aproveitamento
- Eficiência baixa indica overhead, comunicação excessiva ou trabalhadores ociosos

No caso do MPI, a eficiência pode cair quando existem muitas mensagens sendo enviadas, pois a comunicação entre líder e trabalhadores também tem

custo. Assim, tarefas muito pequenas podem gerar overhead de comunicação. Por outro lado, tarefas muito grandes podem causar desequilíbrio de carga, já que alguns trabalhadores podem receber blocos mais pesados que outros. Portanto, o ideal é encontrar um tamanho de tarefa intermediário.

2.4. Garantia De Que Não Haverá Deadlock

Deadlock ocorre quando dois ou mais processos ficam esperando mensagens uns dos outros, mas nenhuma dessas mensagens chega. Para evitar esse problema, o programa foi organizado com um fluxo bem definido de comunicação.

O trabalhador sempre começa esperando uma mensagem do líder. Essa mensagem pode ser uma tarefa ou uma ordem de parada. Se receber uma tarefa, ele processa e devolve um resultado. Depois disso, volta a esperar outra mensagem do líder.

O líder, por sua vez, sempre que recebe um resultado de um trabalhador, imediatamente envia uma nova tarefa ou uma mensagem de parada. Dessa forma, nenhum trabalhador fica esperando para sempre.

Além disso, foram usadas tags diferentes para separar os tipos de mensagem:

TAG_TAREFA: usada quando o líder envia uma tarefa;

TAG_RESULTADO: usada quando o trabalhador devolve o resultado;

TAG_PARAR: usada quando o líder informa que não há mais tarefas.

Portanto, o programa evita deadlock porque todo trabalhador que entra em MPI_Recv recebe obrigatoriamente uma tarefa ou uma mensagem de parada, e o líder sabe exatamente quantos resultados precisa receber.

2.5. Etapas Para Resolução Da Tarefa

- 1º) Criar a função eh_primo;
- 2º) Criar a função para contar primos em um intervalo;
- 3º) Definir o processo de rank 0 como líder;
- 4º) Definir os demais processos como trabalhadores;
- 5º) Dividir o intervalo total em tarefas menores;
- 6º) Fazer o líder enviar tarefas dinamicamente;
- 7º) Fazer cada trabalhador processar uma tarefa por vez;
- 8º) Fazer o trabalhador devolver o resultado ao líder;
- 9º) Calcular o tempo sequencial e o tempo paralelo;
- 10º) Calcular speedup e eficiência;
- 11º) Testar aumentando a quantidade de trabalhadores;
- 12º) Testar aumentando a quantidade de tarefas;
- 13º) Analisar os resultados.

2.6. RESOLUÇÃO EM CÓDIGO DA TAREFA

Segue no apêndice.

2.7. OUTPUT

Segue no apêndice.

3. CONCLUSÃO

A tarefa demonstrou como funciona um escalonador dinâmico utilizando MPI no modelo Líder-Trabalhador. O processo líder ficou responsável por controlar a distribuição das tarefas, enquanto os trabalhadores ficaram responsáveis por calcular a quantidade de números primos em cada intervalo recebido.

O uso do escalonamento dinâmico foi importante porque o cálculo de números primos não possui custo exatamente igual para todos os valores. Assim, ao enviar novas tarefas apenas quando o trabalhador termina a tarefa anterior, o programa consegue distribuir melhor a carga entre os processos.

Em relação ao desempenho, o speedup tende a aumentar conforme mais trabalhadores são adicionados, principalmente quando ainda existe trabalho suficiente para todos. No entanto, a eficiência pode diminuir quando há muitos trabalhadores ou quando as tarefas são pequenas demais, pois o custo de comunicação passa a pesar mais na execução.

Portanto, é possível concluir que o MPI permite paralelizar o problema de forma eficiente, mas o desempenho depende diretamente da quantidade de trabalhadores, do tamanho das tarefas e do custo de comunicação. Além disso, o programa evita deadlock através do uso de mensagens bem definidas e da tag de parada, garantindo que todos os trabalhadores sejam encerrados corretamente ao final da execução.

4. APÊNDICE

4.1. RESOLUÇÃO

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

// essa função vê se o número é primo ou não
int eh_primo(Long Long n) {
    if (n < 2) {
        return 0;
    }

    if (n == 2) {
        return 1;
    }

    if (n % 2 == 0) {
        return 0;
    }

    // aqui eu testo só os ímpares, pq os pares já foram tirados
    for (Long Long i = 3; i <= n / i; i += 2) {
        if (n % i == 0) {
            return 0;
        }
    }
}
```

```

    return 1;
}

// essa função conta quantos primos tem entre inicio e fim
long long contar_primos(long long inicio, long long fim) {
    long long contador = 0;

    for (long long i = inicio; i <= fim; i++) {
        if (eh_primo(i)) {
            contador++;
        }
    }

    return contador;
}

int main(int argc, char *argv[]) {
    int rank, total_processos;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &total_processos);

    // valores padrão caso eu rode sem passar nada no terminal
    long long limite = 1000000;
    long long tamanho_tarefa = 10000;

    // se passar o limite no terminal, ele troca o valor padrão
    if (argc >= 2) {
        limite = atoll(argv[1]);
    }

    // se passar o tamanho da tarefa no terminal, ele troca também
    if (argc >= 3) {
        tamanho_tarefa = atoll(argv[2]);
    }

    // precisa ter pelo menos 1 líder e 1 trabalhador
    if (total_processos < 2) {
        if (rank == 0) {
            printf("Erro: precisa rodar com pelo menos 2 processos.\n");
            printf("Exemplo: mpiexec -n 2 tarefa16.exe 1000000 10000\n");
        }

        MPI_Finalize();
        return 1;
    }
}

```

```

// rank 0 vai ser o líder
if (rank == 0) {
    long long total_seq = 0;
    long long total_par = 0;

    double inicio_seq, fim_seq, tempo_seq;
    double inicio_par, fim_par, tempo_par;

    // primeiro faço a versão sequencial pra ter comparação
    inicio_seq = MPI_Wtime();

    total_seq = contar_primos(2, limite);

    fim_seq = MPI_Wtime();
    tempo_seq = fim_seq - inicio_seq;

    // agora começa a parte paralela
    inicio_par = MPI_Wtime();

    long long proximo_inicio = 2;
    int trabalhadores_ativos = 0;

    // manda uma primeira tarefa pra cada trabalhador
    for (int trabalhador = 1; trabalhador < total_processos;
trabalhador++) {
        long long tarefa[2];

        if (proximo_inicio <= limite) {
            tarefa[0] = proximo_inicio;
            tarefa[1] = proximo_inicio + tamanho_tarefa - 1;

            if (tarefa[1] > limite) {
                tarefa[1] = limite;
            }

            // envia a tarefa para o trabalhador
            MPI_Send(tarefa, 2, MPI_LONG_LONG, trabalhador, 0,
MPI_COMM_WORLD);

            proximo_inicio = tarefa[1] + 1;
            trabalhadores_ativos++;
        } else {
            // se não tiver mais tarefa, já manda parar
            tarefa[0] = -1;
            tarefa[1] = -1;

```



```

        MPI_Send(tarefa, 2, MPI_LONG_LONG, trabalhador, 0,
MPI_COMM_WORLD);
    }
}

// enquanto tiver trabalhador fazendo alguma coisa
while (trabalhadores_ativos > 0) {
    MPI_Status status;
    long long resposta;

    // recebe resultado de qualquer trabalhador que terminar
primeiro
    MPI_Recv(&resposta, 1, MPI_LONG_LONG, MPI_ANY_SOURCE, 1,
MPI_COMM_WORLD, &status);

    int trabalhador_livre = status.MPI_SOURCE;

    total_par += resposta;

    long long tarefa[2];

    if (proximo_inicio <= limite) {
        tarefa[0] = proximo_inicio;
        tarefa[1] = proximo_inicio + tamanho_tarefa - 1;

        if (tarefa[1] > limite) {
            tarefa[1] = limite;
        }

        // manda uma nova tarefa pro trabalhador que acabou de
terminar
        MPI_Send(tarefa, 2, MPI_LONG_LONG, trabalhador_livre, 0,
MPI_COMM_WORLD);

        proximo_inicio = tarefa[1] + 1;
    } else {
        // se não tiver mais nada, manda ele parar
        tarefa[0] = -1;
        tarefa[1] = -1;

        MPI_Send(tarefa, 2, MPI_LONG_LONG, trabalhador_livre, 0,
MPI_COMM_WORLD);

        trabalhadores_ativos--;
    }
}

fim_par = MPI_Wtime();

```

```

tempo_par = fim_par - inicio_par;

int trabalhadores = total_processos - 1;

long long qtd_tarefas = 0;
if (limite >= 2) {
    qtd_tarefas = ((limite - 2 + 1) + tamanho_tarefa - 1) /
tamanho_tarefa;
}

double speedup = tempo_seq / tempo_par;
double eficiencia = (speedup / trabalhadores) * 100.0;

printf("\nRESULTADO\n");
printf("Limite: %lld\n", limite);
printf("Tamanho da tarefa: %lld\n", tamanho_tarefa);
printf("Quantidade de tarefas: %lld\n", qtd_tarefas);
printf("Processos MPI: %d\n", total_processos);
printf("Trabalhadores: %d\n", trabalhadores);

printf("\nVERSAO SEQUENCIAL\n");
printf("Primos encontrados: %lld\n", total_seq);
printf("Tempo: %f segundos\n", tempo_seq);

printf("\nVERSAO PARALELA MPI\n");
printf("Primos encontrados: %lld\n", total_par);
printf("Tempo: %f segundos\n", tempo_par);

printf("\nCOMPARACAO\n");
printf("Resultados iguais? %s\n", total_seq == total_par ? "SIM" :
"NAO");
printf("Speedup: %.2fx\n", speedup);
printf("Eficiencia: %.2f%%\n", eficiencia);
}

// todos os outros ranks são trabalhadores
else {
    while (1) {
        long long tarefa[2];

        // trabalhador fica esperando o líder mandar tarefa
        MPI_Recv(tarefa, 2, MPI_LONG_LONG, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

        // se recebeu -1, é pq acabou tudo
        if (tarefa[0] == -1) {
            break;
        }
    }
}

```

```

    long long inicio = tarefa[0];
    long long fim = tarefa[1];

    // calcula a quantidade de primos da tarefa recebida
    long long qtd_primos = contar_primos(inicio, fim);

    // manda o resultado de volta pro líder
    MPI_Send(&qtd_primos, 1, MPI_LONG_LONG, 0, 1, MPI_COMM_WORLD);
}
}

MPI_Finalize();

return 0;
}

```

4.2. OUTPUT

```
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa16> mpiexec -n 2 tarefa16.exe 1000000 10000
```

```
RESULTADO
Limite: 1000000
Tamanho da tarefa: 10000
Quantidade de tarefas: 100
Processos MPI: 2
Trabalhadores: 1
```

```
VERSAO SEQUENCIAL
Primos encontrados: 78498
Tempo: 0.156895 segundos
```

```
VERSAO PARALELA MPI
Primos encontrados: 78498
Tempo: 0.159168 segundos
```

```
COMPARACAO
Resultados iguais? SIM
Speedup: 0.99x
Eficiencia: 98.57%
```

```
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa16> mpiexec -n 3 tarefa16.exe 1000000 10000
```

```
RESULTADO
Limite: 1000000
Tamanho da tarefa: 10000
Quantidade de tarefas: 100
Processos MPI: 3
Trabalhadores: 2
```

```
VERSAO SEQUENCIAL
Primos encontrados: 78498
Tempo: 0.166548 segundos
```

```
VERSAO PARALELA MPI
Primos encontrados: 78498
Tempo: 0.080300 segundos
```

```
COMPARACAO
Resultados iguais? SIM
Speedup: 2.07x
Eficiencia: 103.70%
```

```
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa16> mpiexec -n 5 tarefa16.exe 1000000 10000
```

```
RESULTADO
Limite: 1000000
Tamanho da tarefa: 10000
Quantidade de tarefas: 100
Processos MPI: 5
Trabalhadores: 4
```

```
VERSAO SEQUENCIAL
Primos encontrados: 78498
```

```
VERSAO SEQUENCIAL
Primos encontrados: 78498
Tempo: 0.239467 segundos
```

```
VERSAO PARALELA MPI
Primos encontrados: 78498
Tempo: 0.089264 segundos
```

```
COMPARACAO
Resultados iguais? SIM
Speedup: 2.68x
Eficiencia: 67.07%
```

```
PS C:\Dados_Z\Documents\UFRN\TI\Período 5\CAD\tarefa16> |
```